Engineering Applications of Artificial Intelligence 142 (2025) 109870

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Research paper

Path planning via reinforcement learning with closed-loop motion control and field tests

Árpád Fehér [a], Ádám Domina [b], Ádám Bárdos [b], Szilárd Aradi [a], Tamás Bécsi [a,*]

[a] Department of Control for Transportation and Vehicle Systems, Faculty of Transportation Engineering and Vehicle Engineering, Budapest University of Technology and Economics, Műegyetem rkp. 3., Budapest, H-1111, Hungary
[b] Department of Automotive Technologies, Faculty of Transportation Engineering and Vehicle Engineering, Budapest University of Technology and Economics, Műegyetem rkp. 3., Budapest, H-1111, Hungary

ARTICLE INFO

Keywords:
Vehicle dynamics
Advanced driver assistance systems
Machine learning
Reinforcement learning
Model predictive control

ABSTRACT

Performing evasive maneuvers with highly automated vehicles is a challenging task. The algorithm must fulfill safety constraints and complete the task while keeping the car in a controllable state. Furthermore, considering all aspects of vehicle dynamics, the path generation problem is numerically complex. Hence its classical solutions can hardly meet real-time requirements. On the other hand, single reinforcement learning based approaches only could handle this problem as a simple driving task and would not provide feasibility information on the whole task's horizon. Therefore, this paper presents a hierarchical method for obstacle avoidance of an automated vehicle to overcome this issue, where the geometric path generation is provided by a single-step continuous Reinforcement Learning agent, while a model-predictive controller deals with lateral control to perform a double lane change maneuver. As the agent plays the optimization role in this architecture, it is trained in various scenarios to provide the necessary parameters for a geometric path generator in a one-step neural network output. During the training, the controller that follows the track evaluates the feasibility of the generated path whose performance metrics provide feedback to the agent so it can further improve its performance. The framework can train an agent for a given problem with various parameters. As a use case, it is presented as a static obstacle avoidance maneuver. the proposed framework was tested on an automotive proving ground with the geometric constraints of the ISO-3888-2 test. The results proved its real-time capability and performance compared to human drivers' abilities.

1. Introduction

Path planning and safe maneuvering are essential in designing Advanced Driver Assistance Systems (ADAS) and are fundamental problems for developing automated vehicle functions. These problems have multiple significant requirements. One is the feasibility, i.e., the generated path or trajectory needs to consider the vehicle's parameters and dynamics to let the actual controller follow it with as slight a deviation as possible. The other aspect is computational performance, or in other words, the real-time capability of the method. Considering every nonlinearity in the vehicle model and providing an optimized solution for the planning horizon is a complex task, leading to increased numerical resources used for solution generation. Considering all holonomic and differential constraints, the optimal solution for finding a path is PSPACE-hard, which naturally remains with the inclusion of a speed profile and dynamic constraints according to Reif (1979). This practically means that one cannot find a proven optimal solution for the path planning problem of highly automated vehicles in real time for a considerable distance to be traveled; hence, many practical methods use simplifications or heuristics to achieve suboptimal yet real-time results, as discussed in the related works. The method proposed in this paper uses a neural network based agent trained through multiple situations to learn the one-step generation of the sub-optimal yet feasible and safe path. According to Paden et al. (2016), driverless functions are built on a hierarchical structure where the layers interact. On the top level, route planning generates a high-level route between the vehicle's position and destination. The behavioral layer defines a local motion specification based on the route, surrounding topology, and obstacles. These are strategic short-term decisions, like lane-keeping or changing, car-following, object avoidance, etc. The motion planning layer generates a reference path or trajectory for the given maneuver, and finally, the local feedback control layer drives the vehicle along this predefined path. Naturally, the layers must interact and redesign the

* Corresponding author.
E-mail address: becsi.tamas@kjk.bme.hu (T. Bécsi).

https://doi.org/10.1016/j.engappai.2024.109870
Received 13 March 2023; Received in revised form 8 October 2024; Accepted 11 December 2024
Available online 20 December 2024
0952-1976/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

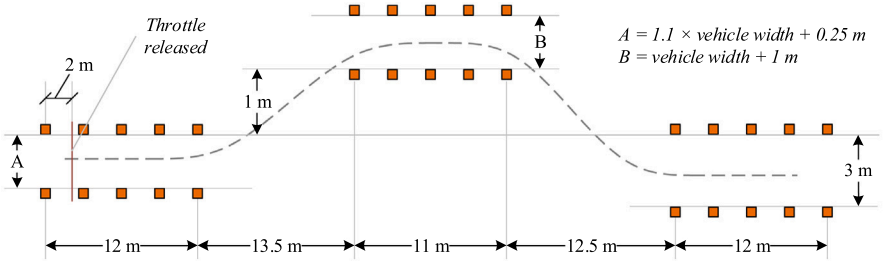


Fig. 1. The double lane-change maneuver according to ISO-3888-2 (International Organization for Standardization (ISO 3888-2:2011), 2011).

motion as the conditions change. The proposed algorithm takes place on the motion planning layer, where it receives a motion primitive and intends to design a feasible path for it. Though many strategic decisions can be made on the behavioral layer, the proposed method is introduced through a double lane-change situation.

A highly automated vehicle must drive the most optimal route when performing a tricky maneuver, such as a double lane change. One can specify several optimization criteria, from comfort to safety (Ciuffo et al., 2021). From all of them, minimizing jerk and lateral acceleration serves both purposes (Hegedűs et al., 2020). Our current research aims to create an experimental method to design the optimal route for a double lane change in real time. Though the trained algorithm can solve the task under a wide range of conditions, a field test is presented with the parameters of the so-called "Moose-test", a double lane-change obstacle avoidance maneuvering setup defined in the standard ISO-3888-2 (International Organization for Standardization (ISO 3888-2:2011), 2011) shown by Fig. 1. During this test, the vehicle accelerates to a defined velocity till the entry point and needs to pass through three straight sections with a released throttle.

As mentioned above, the path is generated by an agent, and an MPC controller performs its following. During experimental tests, two key factors are essential for accurate path tracking: the knowledge of a certain path length ahead of the vehicle and the consideration of the vehicle dynamics in the control law. The Model Predictive Controller (MPC) meets both demands. The first key factor is satisfied by determining the control input by executing an optimization process based on the reference states and the predicted vehicle states. The second constituent is assured by calculating the predicted states based on a simplified vehicle model. Furthermore, the MPC can handle constraints applied as limits for the steering actuator intervention, hence the steering system dynamics have been identified to achieve accurate and reliable path tracking.

1.1. Related work

Path and trajectory planning for highly automated vehicle functions has a broad literature with multiple approaches, best summarized in the following review papers (Paden et al., 2016; Claussmann et al., 2019; González et al., 2015; Ferencz and Zöldy, 2024). The curve, or set of waypoints the vehicle needs to follow, is called a path. If the path is also parametrized by time or velocity, it is called a trajectory. The problem assumes a robot motion model, which can be either holonomic or nonholonomic, kinematic or dynamic, linear or nonlinear, probabilistic or deterministic, etc. Furthermore, it needs to define a path between two endpoints, with given initial conditions, considering static or dynamic obstacles that the robot must avoid. Path planning is widely studied for several types of mobile robots, like underwater vehicles (Ma et al., 2019), ships (Li et al., 2020), Unmanned Aerial Vehicles (Huang and Fei, 2018) or even robot arms (Kunz et al., 2010), all having different dynamics and degrees of freedom. However, as mentioned in the introduction, the high complexity of the complete solution leads to the research of several methods that either neglect dynamic feasibility or aim for a suboptimal solution, using geometrical approaches, heuristics, optimal control, or nonlinear programming. Heuristic or search-based methods usually adopt the A* algorithm and customize it to achieve faster or more feasible paths. For example, in Ferguson et al. (2008), the authors use resampling, while in Likhachev et al. (2005), adaptive refining is proposed. Since the methods hardly conform to the nonholonomic kinematics of the Ackermann steering, some researchers aim to extend the state space to be continuous (Montemerlo et al., 2008). Another search-based approach is the Rapidly-exploring Random Tree (RRT), which considers dynamics and finds a solution in a non-convex space. However, it does not guarantee optimality (Kuwata et al., 2009; Pepy et al., 2006). The fully constrained optimization problem can be dealt with by any solver, though it raises the question of real-timeliness (Lin et al., 2019). One possible approach is to use machine learning to train a neural network for the solutions of many optimization results and use it as an immediate solution or an initial guess for the solver (Hegedüs et al., 2019), which can work for simple setups. However, for more complex scenarios, training dataset generation is also a challenging task to cover the entire state space. Another approach is to use Reinforcement Learning (RL), where the agent interacts with its environment based on trial-and-error and experiences and develops the optimal behavior through the performance metrics, called rewards (Sutton and Barto, 2018). RL is based on the Markov Decision Process, where an agent receives state information from the environment, based on which it generates its action, which alters the environment state by a state transfer function. The environment also provides a so-called reward for each step or at the end of an episode, which is technically an objective function for the task. The agent aims to maximize its reward and learns from the experiences gained from previous episodes. These methods generally utilize end-to-end solutions in the literature, meaning the agent acts with steering and acceleration commands. The trajectory planning is hidden somewhere in the knowledge acquired through the training. As the agent can deal with unstructured data, many sensor models can be used, such as grid-based (Folkers et al., 2019), beam sensors (Lee et al., 2017), camera (Wolf et al., 2017) or ground truth position information (Xia et al., 2016). The other end of the RL-based research focuses on strategic decisions, defining high-level actions, and letting an underlying controller perform the task, which is out of the scope of the paper. Only a few solutions exist in the literature, where the actual path planning is made with RL (Ronecker and Zhu, 2019; Fehér et al., 2020b). One can find a survey on RL-based motion planning in Aradi (2020), and on one of its subsets, imitation learning in Ly and Akhloufi (2020).

Multiple papers focus on hierarchical or multilayer solutions for the combined motion planning problem for different scenarios. The method proposed in Chai et al. (2022b) applies a double-layer structure, where an intelligent optimization method is used in the outer layer such that the main inner optimization routine can be boosted by starting at a better reference solution. The authors demonstrate the approach's effectiveness in a multi-phase overtaking maneuver. To overcome the computational problems of classical methods, the authors of Chai et al. (2022a) utilize a recurrent deep neural network (RDNN) for planning optimal parking maneuver trajectories and an adaptive learning NN-based (ALNN) control scheme for robust trajectory tracking. They also

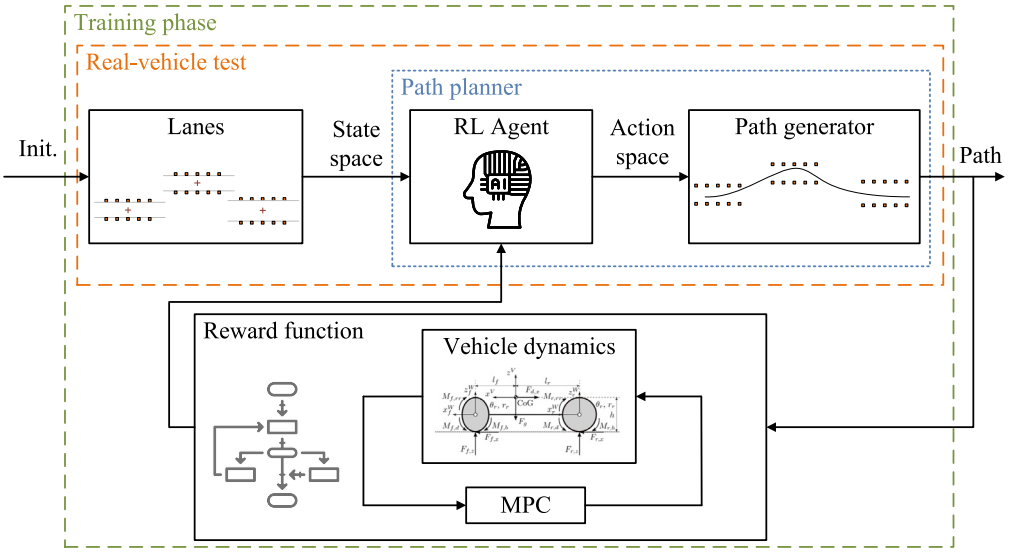


Fig. 2. Training architecture.

propose a multilayer structure in Chai et al. (2022c). In the first layer, a desensitized trajectory optimization method is iteratively performed to establish a set of time-optimal parking trajectories. Then, they train several DNNs to learn the functional relationship between the system state-control actions using this data set.

Motion planning for single or double lane change is also studied by many papers. By using a simple vehicle model, it can be solved as an optimal control problem (Zeping and Jianmin, 2017). A convex approximation is used in Huang et al. (2016) to minimize yaw acceleration on a trajectory that can be dealt with an MPC-based control. Adaptive Neuro-Fuzzy System (ANFIS) is used in Arefnezhad et al. (2018) for a low-speed solution. The authors of Zhou et al. (2017) propose a hyperbolic tangent lane-change trajectory model based on investigating real drivers' maneuvers.

Regarding the trajectory following, three types of MPCs are used for path tracking: the linear parameter-varying (LPV-MPC) (Kianfar et al., 2013; Cao and Zoldy, 2021) the linear time-varying (LTV-MPC) (Falcone et al., 2007), , and the nonlinear (NLMPC) (Borrelli et al., 2005) MPCs. Complexity, performance, accuracy, and the computational requirement of the controllers are increasing, respectively. The LPV-MPC applies a linear system model for state prediction, which does not change over time and has the lowest computational cost. However, it is also the least suitable for handling the nonlinear behavior of the controlled system. In the LTV-MPC, the nonlinear equations describing the system's dynamics are linearized at every timestep in the system's current state. The NLMPC calculates the evolution matrices of the system without linearization generating a nonlinear optimization problem. Hence this is the computationally most expensive method, which may limit the application of this method on real-time systems, as discussed in Falcone et al. (2007).

1.2. Contributions of the paper

The paper presents a reinforcement learning-based path planning solution to determine the path of an emergency double lane change maneuver. The path is made up of polynomial segments whose coefficients are determined by a Twin Delayed Deep Deterministic Policy Gradient (TD3) learning agent. The planner was evaluated by field tests using a moose-test defined by ISO 3888-2. The article details the experimental setup and the MPC controller used for path following. The paper also presents field test measurements of a solution made using a previously published (Fehér et al., 2020a), clothoid-based double lane change path planner. A brief video abstract can be found following the link in the footnote.[1]

Section 2 introduces the RL training method. It describes the training environment, the TD3 agent, the dynamic vehicle model briefly, and solution to generate the polynomial path. In addition, the network architecture, the state, action-space, and reward function are also presented. The following sections are focused on field tests. Section 3 presents the experimental setup. Afterward, Section 4 details the MPC controller used on the tests. Finally, Section 5 provides information on training results, controller, and vehicle behavior on field tests comparing two path planning methods and human drivers.

2. Proposed method and training framework

The task outlined in the previous chapter is solved by combining a reinforcement learning agent and geometrical path planning. As Fig. 2 shows, a learning agent has been placed in a simulation training environment where it needs to learn to act correctly. The reward function determines the correct actions. An episode consists of one step, reducing the task's complexity. In each episode, the agent gets the state-space parameters defining the double lane-changing mission. These parameters are the width, length, and position of the lanes the vehicle must follow without error. Based on this, the agent predicts coefficients (action-space) from which a path is generated. The path is a cubic spline constructed from piecewise third-order polynomials. According to the reviews mentioned earlier (Claussmann et al., 2019; González et al., 2015), for vehicle path planning, polynomial degrees of the splines range from quadratic to quintic, though the most common approaches use cubic splines. Second degree polynomials result in high jerks, while higher grades can be smoother, though computationally more complex. The reward function is responsible for evaluating the goodness of the path. As this motion planning method would not contain feedback, the training loop is closed by an evaluation phase, where a closed-loop MPC controller (see Section 4) drives a dynamic vehicle model along the track while evaluating feasibility. An essential consideration in developing the learning environment is that its outcome should be tested in an actual vehicle. Therefore, it also performs a one-step path planning and follows this path with the closed-loop MPC based on sensor data.

The motivation for applying the proposed combination in a hierarchical solution comes from the fact that the classical methods suffer

[1] Video Abstract: https://youtu.be/xOy9q32PzgA.

from providing feasibility information of a generated path or fail in the real-time requirements. First, an end-to-end RL agent would provide steering input at every control step, though it will not predict the entire path's feasibility. Though training an RL agent with an actor and a value function can not only be used to define a path with its actor, it also provides an estimate of the feasibility of the generated path through its value function. This could be beneficial in safety-related applications, such as driving. On the other hand, if one would like to solve the problem with a standalone MPC, it should incorporate the obstacles as hard constraints for the entire horizon, resulting in a highly complex nonlinear optimization problem unsolvable in real-time. Naturally, one cannot miss out on the computational requirements of the entire issue, which is included in the training phase of the agent.

2.1. Reinforcement learning environment

2.1.1. Agent

Twin-Delayed Deep Deterministic Policy Gradient (TD3) (Fujimoto et al., 2018) was chosen as the agent, which is the successor to Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2015). DDPG is a robust algorithm for continuous action and state-space problems, so it can also efficiently solve automated vehicle control tasks in Fehér et al. (2020a) and Fehér et al. (2019). However, for more complex tasks with large action space, DDPG is very sensitive to hyperparameters (e.g., learning rate, Ornstein–Uhlenbeck noise process parameters, etc.), which can cause convergence to a local maximum.

The DDPG algorithm was presented in detail in our previous article (Fehér et al., 2020a). From now on, the differences are in focus. The better performance of the TD3 compared to the DDPG is provided by three enhancements. The first is the delayed update of the actor-network. However, in actor-critic solutions, the overestimation can cause divergence. TD3 solves this by updating the weights of the actor-network only every second step to reduce the error, resulting in a more stable and efficient training process. Secondly, the algorithm uses two critic networks, with a clipped double Q learning method (Hasselt, 2010). The smaller of the two bad critic networks is selected. This method helps to eliminate underestimation bias. The third method is action noise regularization, which can smooth the target policy. A clipped noise is added to the selected action, which favors higher values for the action, making it more robust.

2.1.2. Network architecture and hyperparameters

To understand the TD3 algorithm, Fig. 3 shows the training process. An experience replay buffer, two actors (target, main), and four critics (main 1, main 2, target 1, and target 2) networks are created to initialize the agent. The algorithm samples from the replay buffer during the training process, where s is the current state, s' is the next state after the action, and r is the reward value. At the beginning of the training, the target network predicts from the s' (next state), a' (next action) to which noise is applied and clamps it in a range of values supported by the environment (according to the Table 1). The critic target networks use the pairs s', a' to predict Q_{t1} and Q_{t2} values. The minimum of these two Q-values represents the approximated value of the next state. The Bellman equation gives the final target value Q_t, where γ is the discount factor. The two main critic networks predict a Q value (Q_1, Q_2) based on (a, s). After that, the loss can be calculated. The critic loss is backpropagated to update the parameters of the two critic models with an SGD optimizer. The actor main can be updated by performing gradient ascent. In every second iteration, the weights of the actor and critic targets are updated by Polyak averaging, meaning the target parameters are updated with a ratio of the main network, which provides smoothing in the training process.

The structure of the couples is the same. Hyper-parameters were determined empirically in addition to the parameters tightly defined by the type of the agent. Though training was stable with the commonly used hyperparameters, the adjustment was done by selecting

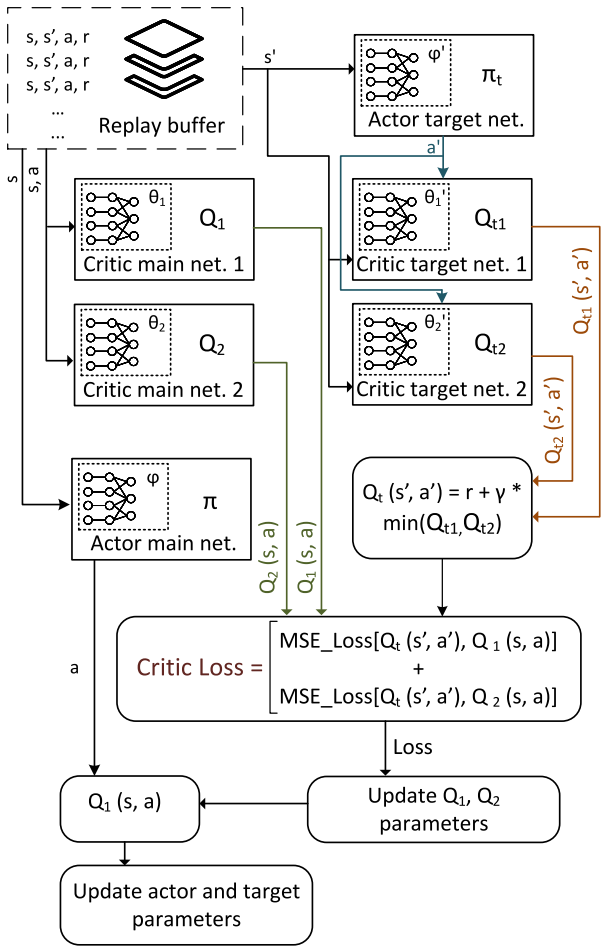


Fig. 3. TD3 training process.

Table 1
Hyperparameters of the training.

Soft-update parameter (τ)	0.005
Batch size	64
Batch selection method	Random choice
Warmup steps	200
Actor main, target networks	
Learning rate (α_a)	0.001
Hidden F.C. layer structure	[1024,512,9]
Update steps	2
Critic main, target networks	
Learning rate (α_c)	0.002
Discount factor (γ)	0.99
Hidden F.C. layer structure	[512, 512, 1]
Action and Target-action noise	
Mean (μ)	0.0
Stddev (σ)	0.3
Target action clip values	−0.5, 0.5

boundaries one scale up and down for them and evaluating the speed of convergence, stability, and average reward. Table 1 summarizes the final hyperparameters.

The first layer of the actor-network is the 11-element state space, followed by two hidden dense layers with a ReLu activation function. Finally, the last layer has nine elements according to the action space with a tanh activation function. The input layer of the critic network consists of concatenating the state and the action space. Two dense

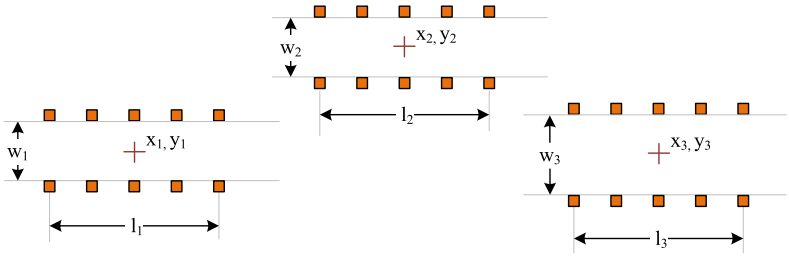


Fig. 4. State representation of the training environment.

layers follow this with a ReLu activation function. Finally, the output layer consists of a single neuron with linear output.

2.1.3. State-space

The state-space consists of 11 continuous elements, describing the lanes (Fig. 4) the vehicle must traverse to complete the maneuver. A rectangle defines each lane. The center points x, y, widths w, and lengths l are included in the state space.

The state-space also includes the vehicle speed v_0 specified at the torque release point:

$$[v_0, l_1, w_1, x_2, y_2, l_2, w_2, x_3, y_3, l_3, w_3]\,. \tag{1}$$

The center of the first lane is always in origin, so it is not part of the state space. The state space elements are normalized to the range of $[0, 1]$ to improve the stability and performance of the training.

2.1.4. Action-space

The action space is a nine-element tuple of real numbers normalized to $[-1, 1]$, and serves as the length and curvature inputs for the polynomial path generator with multiplicative constants. The size of the action space significantly increases the complexity of the problem. In the training process, exploration noise is added to the actions. The path generation inputs are calculated from the action values:

$$[s_1, s_2, s_3, s_4, s_5, \kappa_2, \kappa_3, \kappa_4, \kappa_5, \kappa_6], \text{ where} \tag{2}$$

s_i values stand for the arc lengths, and κ values are the curvature parameters.

2.1.5. Reward function

The reward function implemented for this training differs from the commonly used solutions. Usually, an episode consists of several steps, and a reward value is calculated after each step. The agent receives a reward value after each episode in a one-step case. This one-step training reduces the complexity of the learning task, so it is possible to use a more resource-intensive reward function. The training uses a nonlinear single-track vehicle model containing a dynamic wheel model introduced in Hegedüs et al. (2017), which gives a good trade-off between accuracy and computational complexity. The vehicle model is driven along the track with a lateral MPC controller. The reward is calculated from the performance of the whole loop. A PI controller is responsible for accelerating the vehicle to speed v_0, and during the execution of the maneuver, there is no longitudinal control from the torque release point. According to the reward function, the following events cause the termination of an episode. The agent receives a negative reward (penalty) in these cases.

- Longitudinal rear or front slip greater than 0.1
- Lateral rear or front slip greater than 0.2 radian
- The euclidean distance between vehicle CoG and the nearest point of the path (distance error) greater than 1 meter
- The angle difference between at closest point of the path and vehicle (angle error) greater than 20 degree
- The vehicle hit a cone

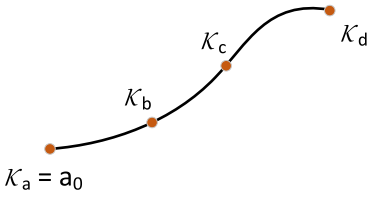


Fig. 5. Curvature parameters of a single polynomial.

- The vehicle does not cross the lane or leaves it sideways

If one of these cases occurs before the last lane, the reward value is −1. In the last lane or after, the reward is −0.75. In case the vehicle passes the test, the reward is calculated as follows:

$$\mu_{max} = 0.0037 e^{v_0\, 0.0693} \tag{3}$$

$$reward_{\mu} = 2\mu_{max} - max(\mu_{lf}) - max(\mu_{lr}), \tag{4}$$

where μ_{max} is a speed-dependent term to enable slightly larger lateral slips at higher speeds, whose parameters are fitted to the typical slip values of a trial training's speed-dependent outputs, while $max(\mu_{lf})$ and $max(\mu_{lr})$ are the maximums of the lateral slips during the entire maneuver, taken at the front and rear tires, respectively.

$$reward_{\ddot{\kappa}} = c_{\ddot{\kappa}} - abs(max(\ddot{\kappa}(s))) - abs(min(\ddot{\kappa}(s))) \tag{5}$$

Another reward component $reward_{\ddot{\kappa}}$ aims to minimize the jerk by reducing the maximum of the double derivative of the curvature function. The value of $c_{\ddot{\kappa}}$ was determined empirically. s is the arc length.

$$reward = w_{\ddot{\kappa}} reward_{\ddot{\kappa}} + w_{\mu} reward_{\mu} \tag{6}$$

The episode reward value is calculated from the sum of the slip and kappa rewards, where $w_{\ddot{\kappa}}$ and w_{μ} are the weights. Two weights are equally chosen to 0.5.

2.2. Polynomial path generation

The trajectory of the evasive maneuver consists of straight lines and polynomial sections. As shown in Fig. 6, three polynomials are defined between two straight sections, defined by length and curvature:

$$\kappa(s) = a_3 s^3 + a_2 s^2 + a_1 s + a_0 \tag{7}$$

As the agent provides curvature values, polynomial coefficients have to be calculated. The coefficient a_0 equals the curvature value κ_a at the beginning of the polynomial. The coefficients a_1, a_2 and a_3 are determined by analytical curve fitting according to κ_b, κ_c, κ_d. According to Fig. 5 κ_b, κ_c is evenly distributed along the length of the polynomial in the middle. κ_d is the curvature value at the end of the curve.

The arc-length integral of the curvature function $\kappa(s)$ is $\theta(s)$, and its arc-length integral is the position $x_s(s)$, $y_s(s)$. $\theta(s)$ is the angle along

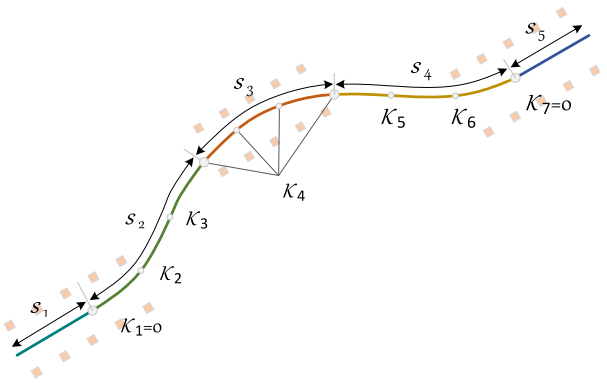


Fig. 6. The path built of polynomial and straight sections.

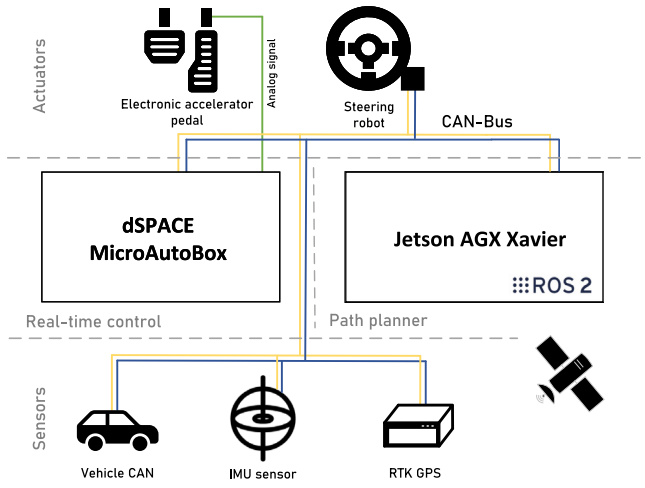


Fig. 7. System architecture of the experimental setup.

the arc length, for which a numerical integration can provide a good approximation.

$$\theta(s) = \theta_0 + \int_0^s a_3 s'^3 + a_2 s'^2 + a_1 s' + a_0 \, ds' \tag{8}$$

$$x_s(s) = x_0 + \int_0^s cos(\theta(s')) \, ds' \tag{9}$$

$$y_s(s) = y_0 + \int_0^s sin(\theta(s')) \, ds' \tag{10}$$

In the action space (2), the length of the straight section according to Fig. 6, the value of one curvature of the second polynomial, the value of two curvatures of the second polynomial, and the value of two curvatures of the third polynomial are included respectively. The length of the last section is fixed.

3. Vehicle and instrumentation setup

As a vehicle platform for the functional field tests, a series production coupé car (BMW M2) was used after suitable modification to enable self-driving.

As shown in Fig. 7, the architecture uses a dSpace MicroAutoBox II 1401 for motion controller implementation, data acquisition, and actuator control. An Nvidia Jetson AGX Xavier development board calculates the AI algorithms to utilize GPU performance. The control loop runs with a 1 ms sample time thanks to the rapid prototyping system.

To enable steering control, a steering robot was installed in the place of the original steering wheel. It was used in angle control mode and communicated via CAN bus with the MicroAutoBox.

For the longitudinal motion control of the car, the accelerator pedal was removed, and the MicroAutoBox emulated its signal. The vehicle's velocity is controlled separately from the lateral motion control by a PI cruise controller to ensure the accurate and reproducible entering speed of the accident avoidance track.

As also shown in Fig. 7, for the self-localization of the vehicle, a GNSS system was used with GSM RTK correction and dual antennas. All states (including sideslip angle, yaw rate, longitudinal velocity, etc.) are calculated from the measured high-frequency position and heading signals.

Due to safety reasons, signals from the vehicle CAN were received by a non-intrusive contactless sensor and were converted to FMS-Standard messages by a special CAN Gateway (Inventure Automotive FMS Gateway). The installation of the instrumentation and the steering robot in the car interior is depicted in Fig. 8.

4. Model predictive controller design

4.1. Vehicle model

The vehicle model applied for state prediction should be selected according to the control objective in the MPC controller. Tracking performance can be significantly better on a higher dimension nonlinear model, though at the price of computational requirements, as seen in Chowdhri et al. (2021). This study uses a linear prediction model to avoid applying successive linearization so that the lateral distance reference is determined in the vehicle's ego coordinate system by transforming the corresponding section of the path using the vehicle's spatial position and orientation.

The orientation error α is the angle between the vehicle's heading direction and the path's tangent. The model used is a linearized lateral dynamic bicycle model with a linear tire model for predicting the states to ensure real-time capability for the control loop. Eq. (11) describes the lateral dynamics of the vehicle. The state vector is $x = [e_1 \, \dot{e}_1 \, e_2 \, \dot{e}_2]^T$, where e_1 is the lateral error and e_2 is the orientation error as defined in (Rajamani, 2012).

$$\dot{x} = Ax + Bu \tag{11}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{2C_{\alpha f}+2C_{\alpha r}}{mV_x} & \frac{2C_{\alpha f}+2C_{\alpha r}}{m} & -\frac{2C_{\alpha f}l_f+2C_{\alpha r}l_r}{mV_x} \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{2C_{\alpha f}l_f-2C_{\alpha r}l_r}{I_z V_x} & \frac{2C_{\alpha f}l_f-2C_{\alpha r}l_r}{m} & -\frac{2C_{\alpha f}l_f^2-2C_{\alpha r}l_r^2}{I_z V_x} \end{bmatrix},$$

$$B = \begin{bmatrix} 0 \\ \frac{2C_{\alpha f}}{m} \\ 0 \\ \frac{2C_{\alpha f}l_f}{I_z} \end{bmatrix},$$

where $C_{\alpha f}$ and $C_{\alpha r}$ are the cornering stiffness of the front and rear tires, m is the vehicle mass, V_x is the velocity, I_x is the vehicle moment of inertia around a given axis, and l_f and l_r are the axis distances from the center of gravity.

Fig. 9 shows the tire characteristics (Bárdos et al., 2020; Czibere et al., 2021) which is nearly linear if the sideslip angle is between 0–3 degrees at the front wheels, and between 0–1.5 degrees at the rear wheels. Hence the prediction is acceptable as long as the value of the sideslip angle is in this range. Accordingly, the MPC needs to execute the path following task up to these limitations of the sideslip, which is confirmed by the experimental test in Section 5.

In numerous researches (Falcone et al., 2007; Xu et al., 2021; Salt Ducajú et al., 2021; Wei et al., 2019) the actuator dynamics are completely neglected. However, it has a high effect on the performance of

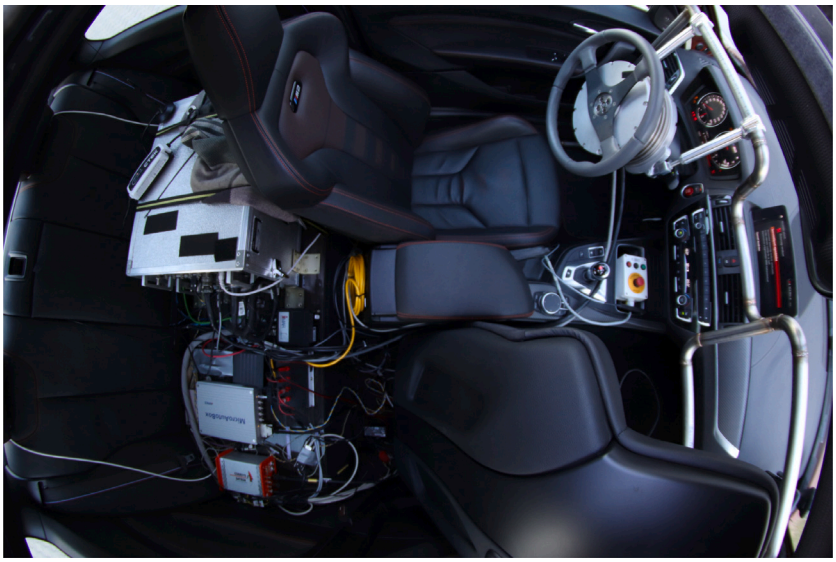


Fig. 8. Instrumentation and steering robot installation in the test vehicle.

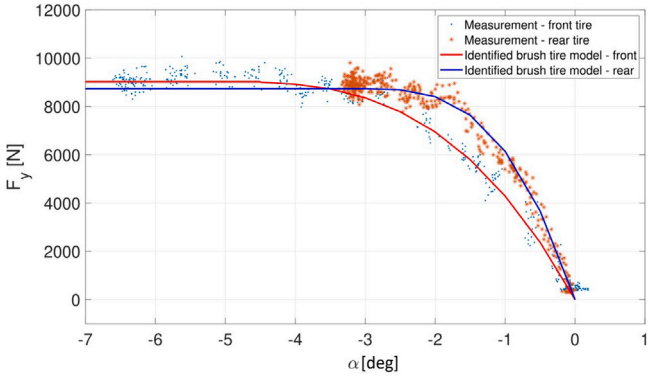


Fig. 9. Front and rear tire characteristics.

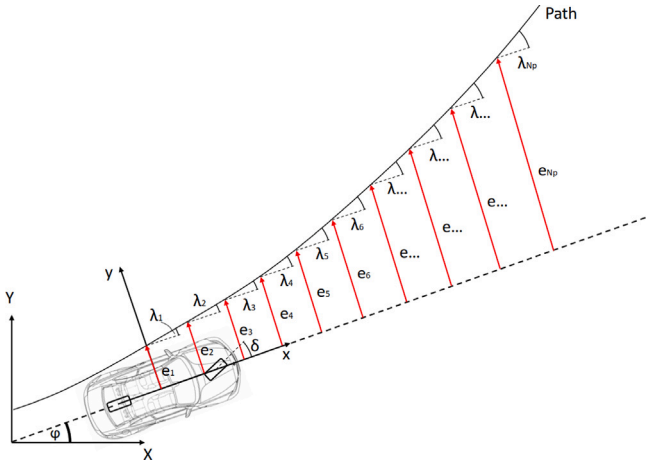


Fig. 10. Reference lateral error and orientation error in the ego frame.

the controller (Domina and Tihanyi, 2019). In this paper, the actuator dynamics is modeled by a first-order linear system (12) as suggested in Bardos et al. (2020). In Kianfar et al. (2013) the dynamics of the steering actuator are also considered this way but are not integrated into the prediction model, only used in the vehicle model.

$$\dot{\delta}_{act} = -\frac{1}{T_{st}}\delta_{act} + \frac{1}{T_{st}}\delta_{ref} \tag{12}$$

In (12), δ_{ref} is the reference steering angle demanded by the controller, δ_{act} is the actual steering angle, and T_{st} is the time constant of the steering system, which is identified from measurement data. Thus, by including the steering dynamics into the vehicle model, a new, modified vehicle model is created (13) as a combination of (12) and (11). The state vector is extended with the actual steering angle δ_{act} and the new state vector is $x = [e_1\ \dot{e}_1\ e_2\ \dot{e}_2,\ \delta_{act}]^T$. According to (13) the model depends on the longitudinal velocity V_x, which can be obtained and used for actual calculations.

$$\dot{x} = Ax + Bu \tag{13}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & -\frac{2C_{\alpha f}+2C_{ar}}{mV_x} & \frac{2C_{\alpha f}+2C_{ar}}{m} & -\frac{2C_{\alpha f}l_f+2C_{ar}l_r}{mV_x} & \frac{2C_{\alpha f}}{m} \\ 0 & 0 & 0 & 1 & 0 \\ 0 & -\frac{2C_{\alpha f}l_f-2C_{ar}l_r}{I_zV_x} & \frac{2C_{\alpha f}l_f-2C_{ar}l_r}{m} & -\frac{2C_{\alpha f}l_f^2-2C_{ar}l_r^2}{I_zV_x} & \frac{2C_{\alpha f}l_f}{I_z} \\ 0 & 0 & 0 & 0 & -\frac{1}{T_{st}} \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \frac{1}{T_{st}} \end{bmatrix}$$

4.2. Reference generation

The reference is defined by a sequence of desired future lateral errors and angular errors in accordance with the states of the applied vehicle plant model (13). Defining the reference path in this way, the nonlinear transformation of the vehicle movement from the ego frame to the global frame can be omitted from the vehicle plant model. Furthermore, the references are defined in a forward-looking way, thereby providing information to the controller on the desired movement of the vehicle. The desired lateral distances and the desired orientation angles look as distance and angular errors from the coordinate system of the vehicle as shown in Fig. 10.

The path is transformed from the global coordinate system to calculate the reference error values into the ego coordinate system. In (14), γ is the rotation angle, E is the offset vector, λ_1 is the orientation error at the closest point to the center of gravity, e_1 is the lateral error at the

closest point to the center of gravity, P_{ref}, i is the transformed point, and P_i is the original path point. According to Fig. 10, (X, Y) is the global frame, (x, y) is the ego frame, the sequence of $e_1, e_2, \dots e_{Np}$ are the desired lateral error references, and $\lambda_1, \lambda_2, \dots \lambda_{Np}$ are the orientation error references.

$$\gamma = \lambda_1 + \varphi \tag{14}$$

$$E = \begin{bmatrix} 0 \\ e_1 \end{bmatrix}$$

$$P_{ref,i} = E + \begin{bmatrix} \cos(\gamma) & -\sin(\gamma) \\ \sin(\gamma) & \cos(\gamma) \end{bmatrix} * P_i$$

The MPC uses Np path points as a horizon for the reference for calculating the optimal sequence of the steering. The method results in a ϕ vector (15) as a reference for each point, where e_i is the reference lateral error for the ith path point and λ_i is the reference orientation error for the ith path point. Finally, the reference vector for the entire prediction horizon is defined by a stacked matrix Φ (16) with $2 \times Np$ rows and one column. Furthermore, the nonlinearities are omitted from the problem definition.

$$\phi = \begin{bmatrix} e_i \\ \lambda_i \end{bmatrix} \tag{15}$$

$$\Phi = [\phi_1\ \phi_2 \dots \phi_{Np}]^T = [e_1\ \lambda_1\ e_2\ \lambda_2 \dots e_{Np}\ \lambda_{Np}]^T \tag{16}$$

4.3. MPC formulation

This section presents the applied control structure. The reference matrix and the vehicle model are updated at every step based on the vehicle's current state. The evolution matrices, the reference matrix, and the vehicle's current state are coupled in a quadratic cost function. The optimal steering demand vector u is calculated after solving the quadratic programming problem.

The evolution matrices are calculated using (17), where A is the state matrix, B is the control input matrix, u is the control input (steering angle), and x is the vector of the state variables.

$$\overbrace{\begin{bmatrix} x(k+1) \\ x(k+2) \\ \vdots \\ x(k+N_p) \end{bmatrix}}^{\hat{\bar{x}}} = \overbrace{\begin{bmatrix} A \\ A^2 \\ \vdots \\ A^{Np} \end{bmatrix}}^{\bar{A}} x(k) + \overbrace{\begin{bmatrix} B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N_p-1}B & A^{N_p-2}B & \dots & B \end{bmatrix}}^{\bar{B}} \overbrace{\begin{bmatrix} u(k) \\ u(k+1) \\ \vdots \\ u(k+N_p-1) \end{bmatrix}}^{\bar{u}} \tag{17}$$

The vehicle states' deviation from the reference states and the magnitude of the applied control input need to be penalized in the cost function. To achieve this, an error vector z is created for punishing the deviation from the reference states (18) where z_1 and z_2 are the error terms for the deviation of the lateral error and the orientation error, respectively.

$$z(k) = \begin{bmatrix} z_1(k) \\ z_2(k) \end{bmatrix} = \begin{bmatrix} e_{1,ref}(k) - e_1(k) \\ e_{2,ref}(k) - e_2(k) \end{bmatrix} = Cx + D\phi = \tag{18}$$

$$\begin{bmatrix} -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} e_1(k) \\ \dot{e_1}(k) \\ e_2(k) \\ \dot{e_2}(k) \\ \delta_{act} \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} e_{1,ref}(k) \\ e_{2,ref}(k) \end{bmatrix}$$

In (18), C is the output matrix, D is a selection matrix for the reference, while $e_{1,ref}(k)$ is the reference lateral error and $e_{2,ref}(k)$ is the reference orientation error. The z error vector can be extended for the entire prediction horizon (19),

$$\overbrace{\begin{bmatrix} z(k+1) \\ z(k+2) \\ \vdots \\ z(k+N_p) \end{bmatrix}}^{\hat{\bar{z}}} = \overbrace{\begin{bmatrix} C & 0 & \dots & 0 \\ 0 & C & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & C \end{bmatrix}}^{\bar{C}} \overbrace{\begin{bmatrix} x(k+1) \\ x(k+2) \\ \vdots \\ x(k+N_p) \end{bmatrix}}^{\hat{\bar{x}}} + \overbrace{\begin{bmatrix} D & 0 & \dots & 0 \\ 0 & D & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & D \end{bmatrix}}^{\bar{D}} \overbrace{\begin{bmatrix} \phi(k+1) \\ \phi(k+2) \\ \vdots \\ \phi(k+N_p) \end{bmatrix}}^{\hat{\Phi}} \tag{19}$$

where both $\bar{C}$ and $\bar{D}$ are stacked diagonal matrices derived from the C and D matrices, $\bar{C} \in R^{2Np \times 5Np}$ is the output hypermatrix, and $\bar{D} \in R^{2Np \times 5Np}$ is the selection hypermatrix for the reference.

In the cost function, the state vector and the error matrix are used to penalize the deviation from the reference states. The control input matrix penalizes the magnitude of the controlled variable. The form of the cost functions for QP (Quadratic Programming) problems is defined as (20) where $\bar{\bar{Q}}_x \in R^{5Np \times 5Np}$ and $\bar{\bar{Q}}_z \in R^{2Np \times 2Np}$ are weight matrices for the state deviation and $\bar{\bar{R}} \in R^{Np \times Np}$ is the weight matrix for the controlled variable, while $\hat{\bar{x}} \in R^{5Np \times Np}$, $\hat{\bar{x}}^T \in R^{Np \times 5Np}$ and $\hat{\bar{z}} \in R^{2Np \times Np}$, $\hat{\bar{z}}^T \in R^{Np \times 2Np}$ are the stacked vectors of the predicted states and errors, respectively, and $\bar{u}^T \in R^{Np \times Np}$ is the optimal control input vector which is the solution of the QP problem.

$$J(k) = \frac{1}{2}(\hat{\bar{x}}^T \bar{\bar{Q}}_x \hat{\bar{x}} + \hat{\bar{z}}^T \bar{\bar{Q}}_z \hat{\bar{z}} + \bar{u}^T \bar{\bar{R}} \bar{u}) \tag{20}$$

$$\bar{\bar{Q}}_x = \begin{bmatrix} Q_x & 0 & \dots & 0 \\ 0 & Q_x & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & Q_x \end{bmatrix} \tag{21}$$

$$\bar{\bar{Q}}_z = \begin{bmatrix} Q_z & 0 & \dots & 0 \\ 0 & Q_z & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & Q_z \end{bmatrix} \bar{\bar{R}} = \begin{bmatrix} R & 0 & \dots & 0 \\ 0 & R & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & R \end{bmatrix}$$

$\bar{\bar{Q}}_x$, $\bar{\bar{Q}}_z$ and $\bar{\bar{R}}$ are diagonal matrices (21) created by using Q_x (22) and Q_z (23) matrices and R scalar (24).

$$Q_x = \begin{bmatrix} q_{x1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & q_{x2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{22}$$

In (22), q_{x1} and q_{x2} are the weights for penalizing the lateral error and the orientation error, respectively. In (23), q_{z1} and q_{z2} are the weights of the deviation from the reference lateral error and the orientation error, respectively, and in (24), r is the weight of the control input.

$$Q_z = \begin{bmatrix} q_{z1} & 0 \\ 0 & q_{z1} \end{bmatrix} \tag{23}$$

$$R = [r] \tag{24}$$

The cost function (25) is calculated by substituting (17) and (18) to (20).

$$J(k) = \frac{1}{2}\bar{u}^T H \bar{u} + f^T \bar{u} \tag{25}$$

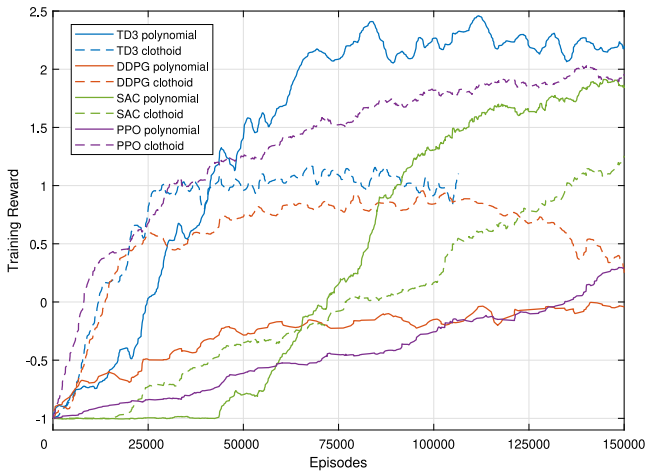


Fig. 11. Comparison of different training setups.

The QP problem is defined in the following form (26), (27):

$$\min_{u} \left[\frac{1}{2} \bar{u}^T H \bar{u} + f^T \bar{u} \right] \tag{26}$$

subject to:

$$-0.5 \leq u_i \leq 0.5 \quad \forall u_i \in \bar{u} \tag{27}$$

The solution to the QP problem is u, which is a $u \in R^{N_p \times 1}$ vector that contains the optimal steering series. In every step, the first element is applied to the vehicle. Then, the measurement of the states, the estimation of future states, and the solution to the QP problem are conducted. This formulation of the MPC leads to a quadratic programming problem with linear constraints, which can be solved in real-time on the MicroAutoBox II, performing this loop in less than 1 ms.

The value of N_p and N_c is 10, the sampling time is 10 ms during the simulations, and the measurements.

5. Results

To better compare the methods and the role of the two concepts in the algorithm's success, an ablation experiment was conducted using the combination of the polynomial- and clothoid-based path generation and the DDPG and TD3 agents. Also, for a wider comparison, Soft Actor-Critic (SAC) (Haarnoja et al., 2018) and Proximal Policy Optimization (PPO) agents (Schulman et al., 2017) were also included.

Each training lasted for 150k episodes, which lasted approximately twenty hours for each training on a medium-performance PC configuration (i7-7700 with GTX 1080 Ti). As shown by Fig. 11, the polynomial-based path planner can perform better than the clothoid-based on this task. However, the DDPG agent was unable to train itself for the polynomial representation, even with several hyperparameters, of which one was shown in the figure. Acceptable reward regarding the clothoid representation is only achieved by the PPO method. Though the PPO-polynomial reached the performance of the TD3-polynomial solution, its training lasted for 500K episodes. SAC-polynomial almost reached the same performance as the TD3. The answer to why the clothoid-based planner is inferior in performance to the polynomial based on comes from the fact that it has a much higher predefined jerk in the mid-section because of the clothoid-straight curve transition, resulting in higher lateral acceleration and the need for stabilization from the side of the controller.

Two questions arise in the preparation of the training environment: One is the so-called sim-to-real problem, and the other one is overfitting an agent for a particular task. As the training environment uses simplified vehicle dynamics and one cannot precisely identify vehicle parameters, the first issue is about robustness in classic control terms, i.e., can the model trained on the environment perform well on the actual plant? The second issue should be that if one defines one direct parameter set for the problem, e.g., the parameters of the Moose-test, the agent will overfit that specific task. However, the design intended to create an agent for the double lane change maneuver under different conditions. The well-known method of domain randomization (Tobin et al., 2017) was chosen to bridge the reality gap of the sim-to-real problem. The environment varies friction parameters and also the length, width, and position of the lanes to be traversed (see Table 2). This also prevents overfitting and provides a more generic double-lane change algorithm. The higher initial speed, the lateral position of the second lane, or the smaller longitudinal distances between the lanes result in a more demanding maneuver in terms of vehicle dynamics. These intervals were chosen so that in the most challenging scenario, even the optimization-based, and hence not real-time AMPC could not fulfill the primary goal, and the vehicle cannot keep the geometric constraints. Naturally, in the more challenging scenarios, the algorithm sacrifices the lateral acceleration and jerk to pass the test, resulting in higher slip values.

Table 2
Training scenario variable ranges.

Parameter	Meaning	Min	Max	Unit
v_0	Initial speed	40	60	km/h
w_1	Lane 1 width	2.2	2.5	m
l_1	Lane 1 length	6	8	m
x_1	Lane 1 center x	3	9	m
w_2	Lane 2 width	2.9	3.9	m
l_2	Lane 2 length	6	6	m
x_2	Lane 2 center x	19	47	m
y_2	Lane 2 center y	0.4	5.2	m
w_3	Lane 3 width	2.5	3.5	m
l_3	Lane 3 length	7	7	m
x_3	Lane 3 center x	35	83	m
y_3	Lane 3 center y	−0.4	−0.2	m
μ	Friction coefficient	0.7	0.95	

The proposed method is compared to a baseline path planning method in simulation. As nonholonomic variants of the A* and RRT algorithms could not provide close-to-optimal solutions in a reasonable time for the given problem, the comparison is carried out with an Adaptive MPC-based path planner. The planner uses a nonlinear kinematic model, where the dynamic constraints are considered by optimizing to steering angle rate of change and lateral acceleration. The same controller drove the generated path as for the proposed method, and the traveled path and lateral acceleration value comparison is shown in Fig. 12.

The performance of the proposed method regarding lateral acceleration is the same as the MPC baseline, though their peaks appear at different points. The proposed planner reaches peak lateral acceleration (and hence, sideslip) at the beginning of the maneuver, while the MPC-based one reaches it when turning back. Though their performance in this way is the same, the MPC needs approximately 1 s of planning time to deal with obstacle avoidance, while the trained agent predicts the parameters in 1 ms and generates the path in 10 ms.

The results were also evaluated with field tests. The measures are performed with the moose test parameter set according to ISO3888-2. The tests were carried out at the ZalaZONE Proving Ground in Zalaegerszeg (Szalay et al., 2018, 2019; Szalay, 2021). In the middle of the dynamic platform, which is an ultra-flat asphalt surface with a diameter of 300 m. The cones were positioned using RTK GPS. The starting position of the vehicle was also determined by fixing a 60 m long straight route for acceleration and a safe distance to braking at the end of the test. The DSC (Dynamic Stability Control for BMW vehicles; can be considered as ESP system) was switched off during the measurements because the effect of the DSC was also neglected in the training environment.

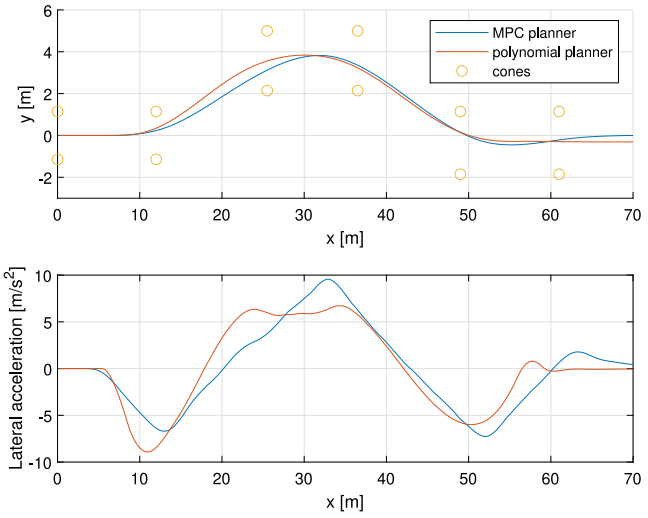


Fig. 12. Comparison of the performance of the polynomial and the pure Adaptive MPC planners in simulation.

Three types of field tests were performed:

- 14 done by 7 human drivers,
- 11 done by the TD3 agent generated polynomial path introduced in this article, performed by the real-time control algorithm,
- 11 done by the DDPG agent generated clothoid path (Fehér et al., 2020a), performed by also the real-time control algorithm.

The tests were carried out under the same conditions for the humans and the algorithms. The geometric layout of the test is presented in Fig. 1 earlier. Longitudinal control, e.g., acceleration to the target and the throttle release at the entry point, was performed by a separate controller to ensure comparability. Hence, only the lateral control was performed by humans and the algorithms by steering. During the tests, the system autonomously accelerated the vehicle to the torque release line to a predetermined initial speed to guarantee reproducibility. The trained AI generated the evasive path, and the MPC controller produced the steering angle demand for the steering robot. During the tests, two supervising drivers were sitting in the car. Measurement data from IMU, RTK GPS, Vehicle CAN, etc., were recorded with 100 Hz sampling.

Seven drivers performed the 14 human tests. The drivers were between the ages of 25 and 43, with driving experience between 1 and 25 years, without any professional training, except for one driver, a Formula Student racing driver, who also performed best amongst the humans in the test. Five human runs were successful without touching a cone. Driving into the entrance lane is already difficult for a human driver with average skills. Finding the optimal path is only possible after many attempts. The other 22 tests were distributed equally between the two agents, which successfully passed in all cases.

The real-time realization of the lateral control was evaluated on MicroAutobox, which is also the target hardware of the test vehicle, and is capable of providing turnaround time for the given algorithm. As Fig. 13 shows, the MPC algorithm itself always stayed under 0.225 ms, and the control loop could keep its real-time capability during the tests.

Fig. 14 shows the successful completion of the moose test at a speed of 55 km/h on the test track. The top diagram shows the cones defined by ISO3888-2, the traversed routes, and the lateral accelerations below. Red represents clothoid-based, blue represents polynomial-based, and the green represents a human driver executed maneuver. The clothoid planner agent builds the trajectory from straight lines and clothoid segments with 0 to 0 curvature changes in the following order: straight-curve-straight-curve-straight. The main disadvantage of this solution

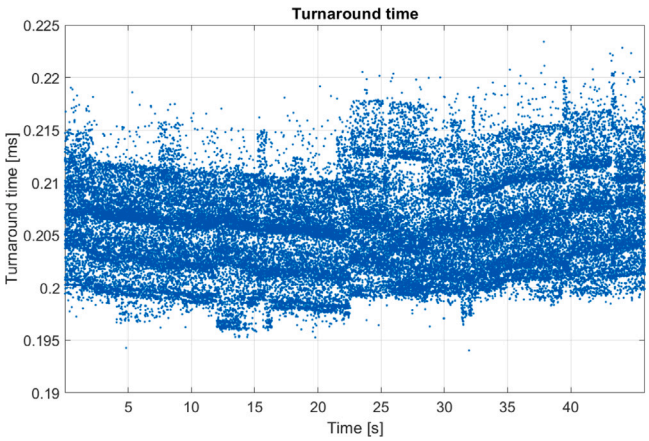


Fig. 13. MicroAutoBox turnaround time measurement of the path following MPC.

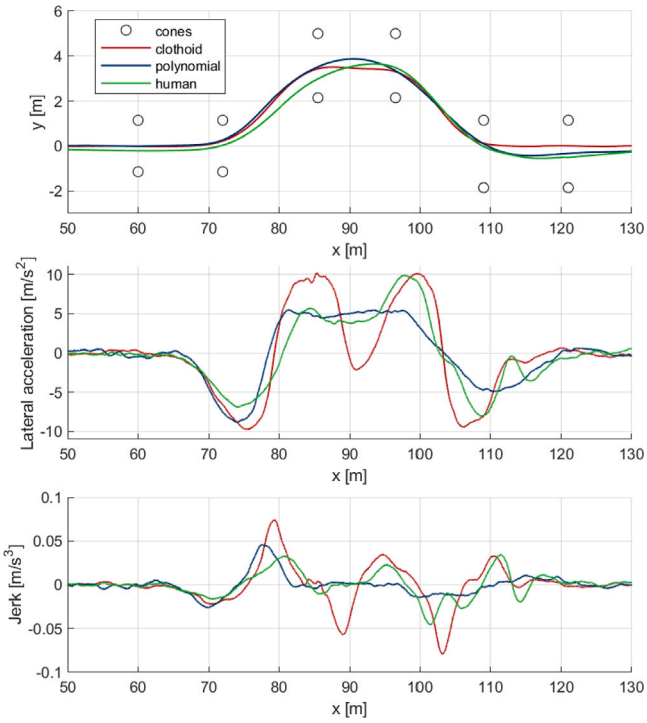


Fig. 14. Traversed routes and lateral accelerations of same initial speed of successful ISO 3888-2 experiments on the test track.

is that the curvature of the path always falls back to 0, causing unnecessary jerk in the middle of the maneuver. Because the polynomial designer uses a constant curvature section instead of a straight section, the curvature of the trajectory does not fall to 0 in the middle, so this route resulted in smaller jerks and greater stability and travel convenience. Fig. 14 also shows that the average human driver can only pass the moose test without error with many corrections.

Fig. 15 shows the results of several runs performed at nearly the same speed. The figure shows that the MPC controller follows the reference path with a slight lateral error almost in every case. The shape of the lateral acceleration diagram of the clothoid path is roughly the same as a function of the curvature of the reference path. Due to the straight section in the middle of the track, the lateral acceleration suddenly falls into the negative range and returns to the maximum value. Due to the large jerks, the vehicle reached the saturation of the tires several times, leading to increased control errors. In the lateral acceleration diagram, the noise at 110 m can be observed for some runs,

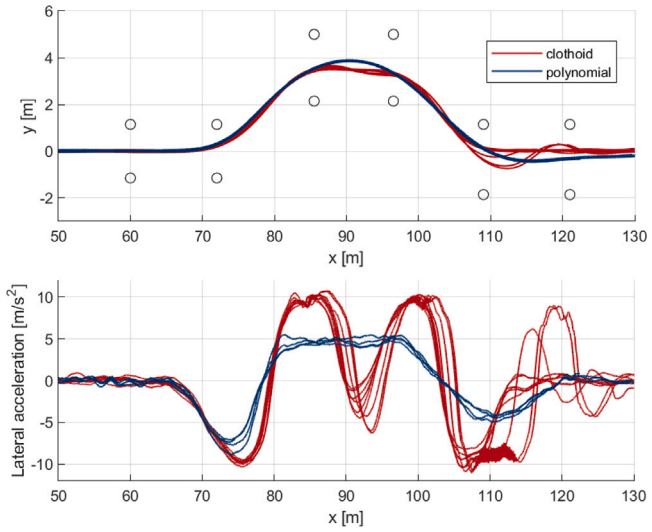


Fig. 15. Path and lateral acceleration comparison of the clothoid and polynomial based methods on the test track.

showing the vibrations resulting from the loss of adhesion. Sometimes, the MPC controller returned the vehicle to the reference route, even in slip events. The diagrams also show that the polynomial-based designer provides a more feasible reference path.

Fig. 16 shows the performance of the path following MPC and the behavior of the vehicle through a moose test performed at an initial speed of 55 km/h using both the clothoid-based and the polynomial paths in simulation, which decreased quasi-linearly by the effect of the engine brake to approximately 45 km/h. The MPC provides better accuracy on the polynomial path, on which the maximum lateral error is 9 cm, and the maximum angular error is 3.5 degrees, while on the clothoid-based path, the maximum lateral error is 11 cm, and the maximum angular error is 5 degrees. The smaller error values clearly show the polynomial path's advantages. Furthermore, the steering angle demand is more smooth on the polynomial path, which results in better vehicle stability and a more comfortable ride.

Concerning the sideslip angles, the MPC remains robust beyond the linear region of the tires, the maximal sideslip angle is 6 degrees at the front wheels and 3 degrees at the rear wheels, while the stability limit is at 3 degrees, and 1.5 degrees, respectively. However, the sideslip angles only temporarily exceeded the linear region of the tire characteristics. The maximal tire sideslip angles are significantly greater when the vehicle is run on the clothoid-based path. At the polynomial path, the sideslip angles are less, which corresponds to fewer lateral accelerations. Accordingly, less sideslip means more stable controller performance because there is more reserve about the sideslip to reach the end of the linear region of the tire characteristics. Overall, the smoother curvatures of the polynomial path result in less lateral acceleration and fewer sideslip angles which provides a more stable, safer, and more comfortable ride.

6. Conclusion

The article presents a real-time geometric path planning agent trained with Reinforcement Learning for performing a double lane change maneuver. The field test showed the success of domain adaptation, as the trained agent successfully performed the test in all cases. Furthermore, it outperformed an earlier clothoid-based designer and human drivers also.

Naturally, there are impossible scenarios where the motion planning task has no solution based on the dynamic capabilities of the vehicle and the geometric parameters of the expected maneuver. In this or any

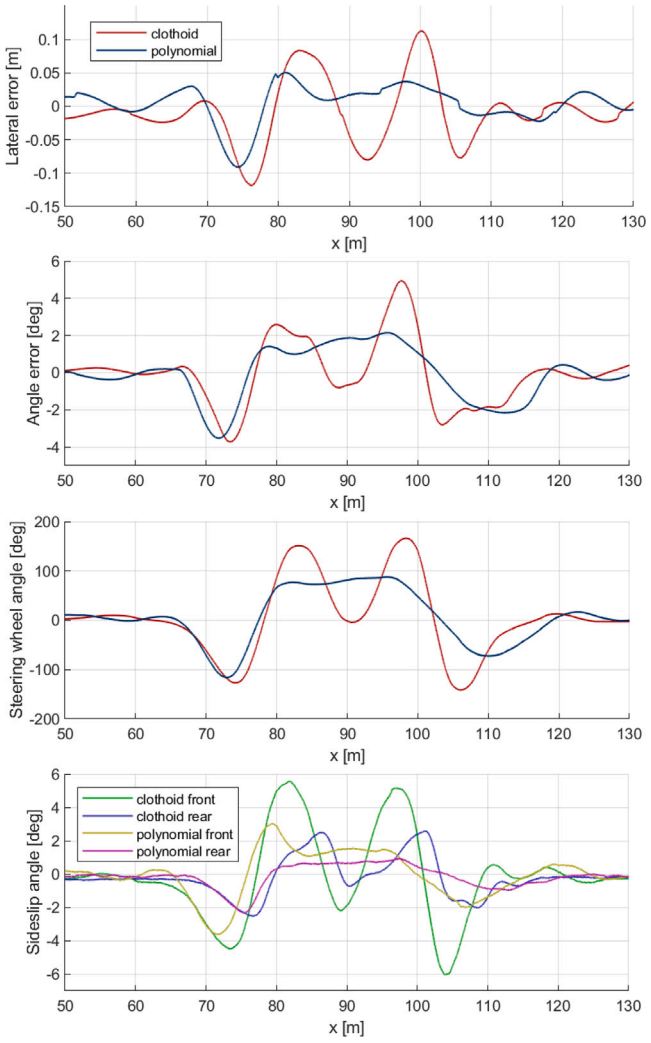


Fig. 16. MPC controller errors, steering wheel and sideslip angles of same initial speed of successful ISO 3888-2 experiments in simulation.

feasible case, one must have information about the feasibility of the designed path for safety reasons. The designed path can be evaluated in two ways. First, both DDPG and TD3 agents consist of an actor and a critic network from which the actor provides the action, which gives the path parameters, while the critic network predicts the Q value of this action. This value estimates the expected reward, which holds information on the feasibility of the maneuver. The other possibility is the model-based verification of the result, where the designed path is evaluated with simulation. When resulting in a detected unfeasible path, this motion planning layer should interact with the upper behavioral layer to choose another strategy.

The benefit of using a hierarchical approach compared to the common end-to-end RL approaches is that the feasibility of the maneuver increases, which the agent's value function can also predict. On the other hand, it also serves as a drawback since this solution needs a prototype maneuver, e.g., a predefined path generator for a particular type of maneuver (the double lane change in our case). Though the provided structure can be trained for various other path generators, giving a maneuver pool from which the upper behavioral motion planning layer could choose the most appropriate one, which could be an interesting future research aim. The presented design works on a static obstacle avoidance problem. However, as the designed agent can act as a one-step optimizer for a path planning problem on a given horizon, it can be used as part of a nonlinear MPC planner, which insists

that it can be trained to evade dynamic objects, such as pedestrians. In this case, the real-time design on the horizon should act as a control input in the loop, with continuous replanning. Our future research will focus on this solution.

CRediT authorship contribution statement

Árpád Fehér: Writing – original draft, Visualization, Software, Investigation, Data curation, Conceptualization. **Ádám Domina:** Writing – original draft, Software, Methodology, Investigation, Data curation. **Ádám Bárdos:** Validation, Methodology, Investigation, Formal analysis. **Szilárd Aradi:** Writing – review & editing, Validation, Supervision, Conceptualization. **Tamás Bécsi:** Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The research was supported by the European Union within the framework of the National Laboratory for Autonomous Systems. (RRF-2.3.1-21-2022-00002)

The research reported in this paper is part of project no. BME-NVA-02, implemented with the support provided by the Ministry of Innovation and Technology of Hungary from the National Research, Development and Innovation Fund, financed under the TKP2021 funding scheme.

Data availability

No data was used for the research described in the article.

References

Aradi, S., 2020. Survey of deep reinforcement learning for motion planning of autonomous vehicles. IEEE Trans. Intell. Transp. Syst. 1–20. http://dx.doi.org/10.1109/TITS.2020.3024655.

Arefnezhad, S., Ghaffari, A., Khodayari, A., Nosoudi, S., 2018. Modeling of double lane change maneuver of vehicles. Int. J. Automot. Technol. 19 (2), 271–279.

Bárdos, Á., Domina, Á., Tihanyi, V., Szalay, Z., Palkovics, L., 2020. Implementation and experimental evaluation of a mimo drifting controller on a test vehicle. In: 2020 IEEE Intelligent Vehicles Symposium. IV, pp. 1472–1478. http://dx.doi.org/10.1109/IV47402.2020.9304820.

Bardos, A., Szimandl, B., Nemeth, H., 2020. Controller structure for high response engine exhaust throttles. Int. J. Heavy Veh. Syst. 27, 325. http://dx.doi.org/10.1504/IJHVS.2020.108736.

Borrelli, F., Falcone, P., Keviczky, T., Asgari, J., Hrovat, D., 2005. Mpc-based approach to active steering for autonomous vehicle systems. Int. J. Veh. Auton. Syst. 3 (2–4), 265–291.

Cao, H., Zoldy, M., 2021. Mpc tracking controller parameters impacts in roundabouts. Mathematics 9 (12).

Chai, R., Liu, D., Liu, T., Tsourdos, A., Xia, Y., Chai, S., 2022a. Deep learning-based trajectory planning and control for autonomous ground vehicle parking maneuver. IEEE Trans. Autom. Sci. Eng. 1–15. http://dx.doi.org/10.1109/TASE.2022.3183610.

Chai, R., Tsourdos, A., Chai, S., Xia, Y., Savvaris, A., Chen, C., 2022b. Multi-phase overtaking maneuver planning for autonomous ground vehicles via a desensitized trajectory optimization approach. IEEE Trans. Ind. Inform. 1. http://dx.doi.org/10.1109/TII.2022.3168434.

Chai, R., Tsourdos, A., Savvaris, A., Chai, S., Xia, Y., Chen, C.L.P., 2022c. Design and implementation of deep neural network-based control for automatic parking maneuver process. IEEE Trans. Neural Netw. Learn. Syst. 33 (4), 1400–1413. http://dx.doi.org/10.1109/TNNLS.2020.3042120.

Chowdhri, N., Ferranti, L., Iribarren, F.S., Shyrokau, B., 2021. Integrated nonlinear model predictive control for automated driving. Control Eng. Pract. 106, 104654. http://dx.doi.org/10.1016/j.conengprac.2020.104654.

Ciuffo, B., Mattas, K., Makridis, M., Albano, G., Anesiadou, A., He, Y., Josvai, S., Komnos, D., Pataki, M., Vass, S., Szalay, Z., 2021. Requiem on the positive effects of commercial adaptive cruise control on motorway traffic and recommendations for future automated driving systems. Transp. Res. C 130, 103305. http://dx.doi.org/10.1016/j.trc.2021.103305.

Claussmann, L., Revilloud, M., Gruyer, D., Glaser, S., 2019. A review of motion planning for highway autonomous driving. IEEE Trans. Intell. Transp. Syst. 21 (5), 1826–1848.

Czibere, S., Domina, Á., Bárdos, Á., Szalay, Z., 2021. Model predictive controller design for vehicle motion control at handling limits in multiple equilibria on varying road surfaces. Energies 14 (20), http://dx.doi.org/10.3390/en14206667.

Domina, Á., Tihanyi, V., 2019. Modelling the dynamic behavior of the steering system for low speed autonomous path tracking. In: 2019 IEEE Intelligent Vehicles Symposium. IV, pp. 535–540. http://dx.doi.org/10.1109/IVS.2019.8814185.

Falcone, P., Borrelli, F., Asgari, J., Tseng, H.E., Hrovat, D., 2007. Predictive active steering control for autonomous vehicle systems. IEEE Trans. Control Syst. Technol. 15 (3), 566–580. http://dx.doi.org/10.1109/TCST.2007.894653.

Fehér, Á., Aradi, S., Bécsi, T., 2020a. Hierarchical evasive path planning using reinforcement learning and model predictive control. IEEE Access 8, 187470–187482. http://dx.doi.org/10.1109/ACCESS.2020.3031037.

Fehér, Á., Aradi, S., Bécsi, T., Gáspár, P., Szalay, Z., 2020b. Proving ground test of a DDPG-based vehicle trajectory planner. In: 2020 European Control Conference. ECC, pp. 332–337.

Fehér, Á., Aradi, S., Hegedüs, F., Bécsi, T., Gáspár, P., 2019. Hybrid ddpg approach for vehicle motion planning. In: Proceedings of the 16th International Conference on Informatics in Control, Automation and Robotics. ICINCO 2019, SciTePress, 1Setúbal, Portugália.

Ferencz, C., Zöldy, M., 2024. Exhaustive investigation of the promises and perils of autonomous mobility technology. Period. Polytech. Transp. Eng. 52 (1), 86–95. http://dx.doi.org/10.3311/PPtr.22573.

Ferguson, D., Howard, T.M., Likhachev, M., 2008. Motion planning in urban environments: Part ii. In: Lab Papers. GRASP, p. 29.

Folkers, A., Rick, M., Buskens, C., 2019. Controlling an Autonomous Vehicle with Deep Reinforcement Learning. IV, IEEE, pp. 2025–2031. http://dx.doi.org/10.1109/IVS.2019.8814124.

Fujimoto, S., van Hoof, H., Meger, D., 2018. Addressing function approximation error in actor-critic methods. In: Proceedings of the 35th International Conference on Machine Learning. Vol. 80, PMLR, pp. 1587–1596, Stockholmsmässan, Stockholm Sweden, URL.

González, D., Pérez, J., Milanés, V., Nashashibi, F., 2015. A review of motion planning techniques for automated vehicles. IEEE Trans. Intell. Transp. Syst. 17 (4), 1135–1145.

Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: Proceedings of the 35th International Conference on Machine Learning. pp. 1861–1870.

Hasselt, H., 2010. Double q-learning. Adv. Neural Inf. Process. Syst. 23, 2613–2621.

Hegedűs, T., Németh, B., Gáspár, P., 2020. Challenges and possibilities of overtaking strategies for autonomous vehicles. Period. Polytech. Transp. Eng. 48 (4), 320–326. http://dx.doi.org/10.3311/PPtr.15848.

Hegedüs, F., Bécsi, T., Aradi, S., Gáspár, P., 2017. Model based trajectory planning for highly automated road vehicles. IFAC-PapersOnLine 50 (1), 6958–6964. http://dx.doi.org/10.1016/j.ifacol.2017.08.1336, 20th IFAC World Congress.

Hegedüs, F., Bécsi, T., Aradi, S., Gáspár, P., 2019. Motion planning for highly automated road vehicles with a hybrid approach using nonlinear optimization and artificial neural networks. Strojniski Vestnik-J. Mech. Eng. 65 (3), 148–160.

Huang, C., Fei, J., 2018. Uav path planning based on particle swarm optimization with global best path competition. Int. J. Pattern Recognit. Artif. Intell. 32 (06), 1859008. http://dx.doi.org/10.1142/S0218001418590085.

Huang, C., Naghdy, F., Du, H., 2016. Model predictive control-based lane change control system for an autonomous vehicle. In: 2016 IEEE Region 10 Conference. TENCON, IEEE, pp. 3349–3354.

International Organization for Standardization (ISO 3888-2:2011), 2011. Passenger Cars — Test Track for a Severe Lane-Change Manoeuvre — Part 2: Obstacle Avoidance, Standard. International Organization for Standardization, Geneva, CH.

Kianfar, R., Falcone, P., Fredriksson, J., 2013. A distributed model predictive control approach to active steering control of string stable cooperative vehicle platoon. IFAC Proc. Vol. 46 (21), 750–755.

Kunz, T., Reiser, U., Stilman, M., Verl, A., 2010. Real-time path planning for a robot arm in changing environments. In: 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 5906–5911. http://dx.doi.org/10.1109/IROS.2010.5653275.

Kuwata, Y., Teo, J., Fiore, G., Karaman, S., Frazzoli, E., How, J.P., 2009. Real-time motion planning with applications to autonomous urban driving. IEEE Trans. Control Syst. Technol. 17 (5), 1105–1118.

Lee, J., Kim, T., Kim, H.J., 2017. Autonomous lane keeping based on approximate Q-learning. In: 2017 14th International Conference on Ubiquitous Robots and Ambient Intelligence. URAI, IEEE, pp. 402–405. http://dx.doi.org/10.1109/URAI.2017.7992762.

Li, G., Hildre, H.P., Zhang, H., 2020. Toward time-optimal trajectory planning for autonomous ship maneuvering in close-range encounters. IEEE J. Ocean. Eng. 45 (4), 1219–1234. http://dx.doi.org/10.1109/JOE.2019.2926822.

Likhachev, M., Ferguson, D., Gordon, G.J., Stentz, A., Thrun, S., 2005. Anytime dynamic a*: An anytime, replanning algorithm. In: ICAPS. pp. 262–271.

Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2015. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971.

Lin, Y., McPhee, J., Azad, N.L., 2019. Longitudinal dynamic versus kinematic models for car-following control using deep reinforcement learning. In: 2019 IEEE Intelligent Transportation Systems Conference. ITSC, IEEE, pp. 1504–1510.

Ly, A.O., Akhloufi, M.A., 2020. Learning to drive by imitation: an overview of deep behavior cloning methods. IEEE Trans. Intell. Veh..

Ma, Y.-N., Gong, Y.-J., Xiao, C.-F., Gao, Y., Zhang, J., 2019. Path planning for autonomous underwater vehicles: An ant colony algorithm incorporating alarm pheromone. IEEE Trans. Veh. Technol. 68 (1), 141–154. http://dx.doi.org/10.1109/TVT.2018.2882130.

Montemerlo, M., Becker, J., Bhat, S., et al., 2008. Junior: The stanford entry in the urban challenge. J. Field Robotics 25 (9), 569–597.

Paden, B., Čáp, M., Yong, S.Z., Yershov, D., Frazzoli, E., 2016. A survey of motion planning and control techniques for self-driving urban vehicles. IEEE Trans. Intell. Veh. 1 (1), 33–55.

Pepy, R., Lambert, A., Mounier, H., 2006. Path planning using a dynamic vehicle model. In: 2006 2nd International Conference on Information & Communication Technologies. Vol. 1, IEEE, pp. 781–786.

Rajamani, R., 2012. Vehicle Dynamics and Control. Springer Science & Business Media, http://dx.doi.org/10.1007/978-1-4614-1433-9.

Reif, J.H., 1979. Complexity of the mover's problem and generalizations. In: 20th Annual Symposium on Foundations of Computer Science. sfcs 1979, pp. 421–427. http://dx.doi.org/10.1109/SFCS.1979.10.

Ronecker, M.P., Zhu, Y., 2019. Deep Q-network based decision making for autonomous driving. In: 2019 3rd International Conference on Robotics and Automation Sciences. ICRAS, IEEE, pp. 154–160. http://dx.doi.org/10.1109/ICRAS.2019.8808950.

Salt Ducajú, J.M., Salt Llobregat, J.J., Cuenca, Á., Tomizuka, M., 2021. Autonomous ground vehicle lane-keeping LPV model-based control: Dual-rate state estimation and comparison of different real-time control strategies. Sensors 21 (4), http://dx.doi.org/10.3390/s21041531.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv:1707.06347.

Sutton, R.S., Barto, A.G., 2018. Reinforcement Learning: An Introduction. MIT press.

Szalay, Z., 2021. Next generation x-in-the-loop validation methodology for automated vehicle systems. IEEE Access 9, 35616–35632. http://dx.doi.org/10.1109/ACCESS.2021.3061732.

Szalay, Z., Hamar, Z., Simon, P., 2019. A multi-layer autonomous vehicle and simulation validation ecosystem axis: Zalazone. In: Advances in Intelligent Systems and Computing. Vol. 867, Springer Verlag, pp. 954–963. http://dx.doi.org/10.1007/978-3-030-01370-7_74.

Szalay, Z., Tettamanti, T., Esztergár-Kiss, D., Varga, I., Bartolini, C., 2018. Development of a test track for driverless cars: Vehicle design, track configuration, and liability considerations. Period. Polytech. Transp. Eng. 46 (1), 29–35. http://dx.doi.org/10.3311/PPtr.10753.

Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P., 2017. Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, pp. 23–30. http://dx.doi.org/10.1109/IROS.2017.8202133.

Wei, S., Zou, Y., Zhang, X., Zhang, T., Li, X., 2019. An integrated longitudinal and lateral vehicle following control system with radar and vehicle-to-vehicle communication. IEEE Trans. Veh. Technol. 68 (2), 1116–1127. http://dx.doi.org/10.1109/TVT.2018.2890418.

Wolf, P., Hubschneider, C., Weber, M., Bauer, A., Hartl, J., Durr, F., Zollner, J.M., 2017. Learning how to drive in a real world simulation with deep Q-networks. In: 2017 IEEE Intelligent Vehicles Symposium. IV, IEEE, pp. 244–250. http://dx.doi.org/10.1109/IVS.2017.7995727.

Xia, W., Li, H., Li, B., 2016. A control strategy of autonomous vehicles based on deep reinforcement learning. In: 2016 9th International Symposium on Computational Intelligence and Design. ISCID, IEEE, pp. 198–201. http://dx.doi.org/10.1109/ISCID.2016.2054.

Xu, Y., Tang, W., Chen, B., Qiu, L., Yang, R., 2021. A model predictive control with preview-follower theory algorithm for trajectory tracking control in autonomous vehicles. Symmetry 13 (3), http://dx.doi.org/10.3390/sym13030381.

Zeping, F., Jianmin, D., 2017. Optimal lane change motion of intelligent vehicles based on extended adaptive pseudo-spectral method under uncertain vehicle mass. Adv. Mech. Eng. 9 (7), 1687814017702823.

Zhou, B., Wang, Y., Yu, G., Wu, X., 2017. A lane-change trajectory model from drivers' vision view. Transp. Res. C 85, 609–627. http://dx.doi.org/10.1016/j.trc.2017.10.013.